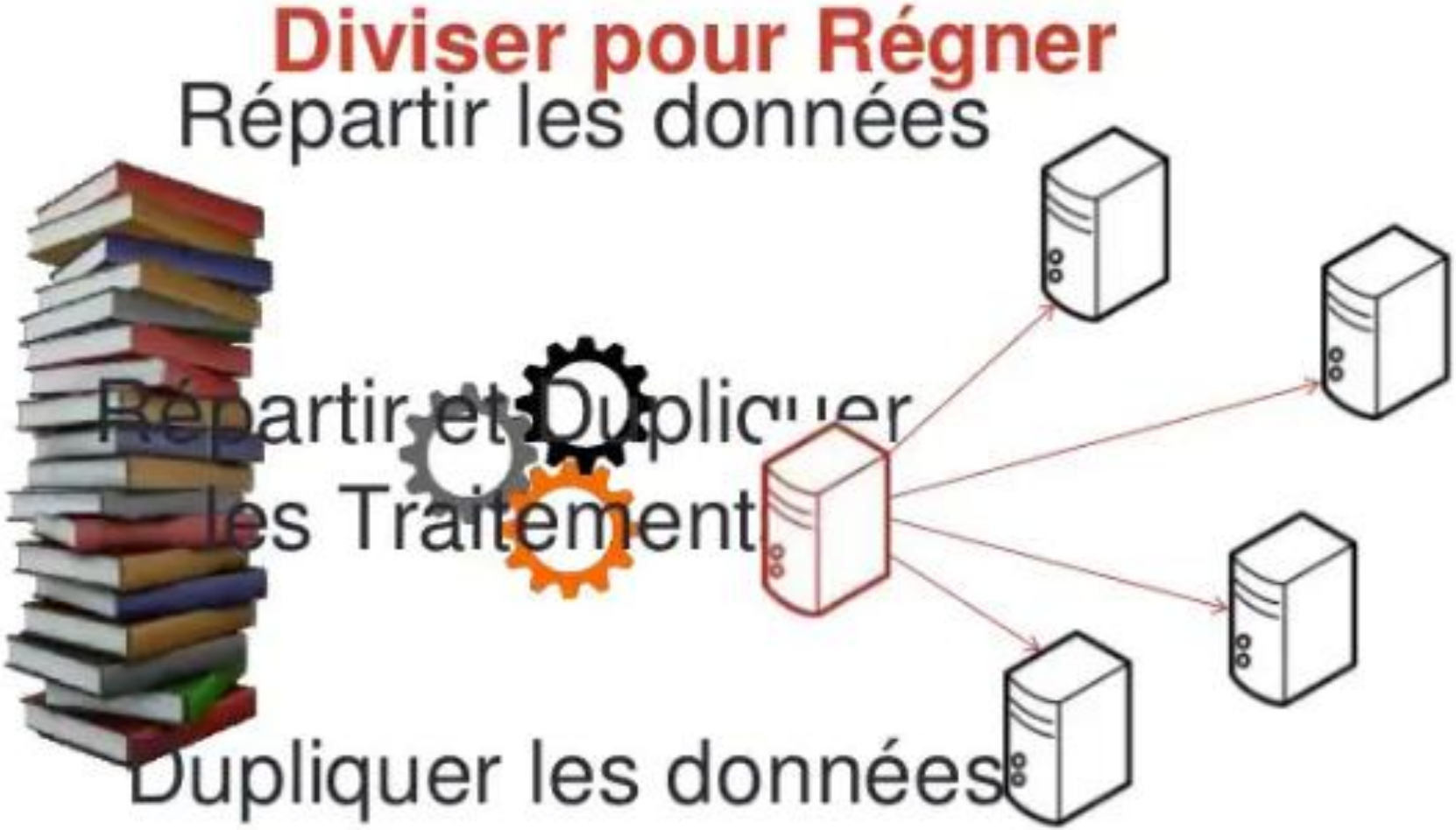
Traitement des Données Massives



Traitement des Données Massives



Traitement des Données Massives

Diviser pour Régner



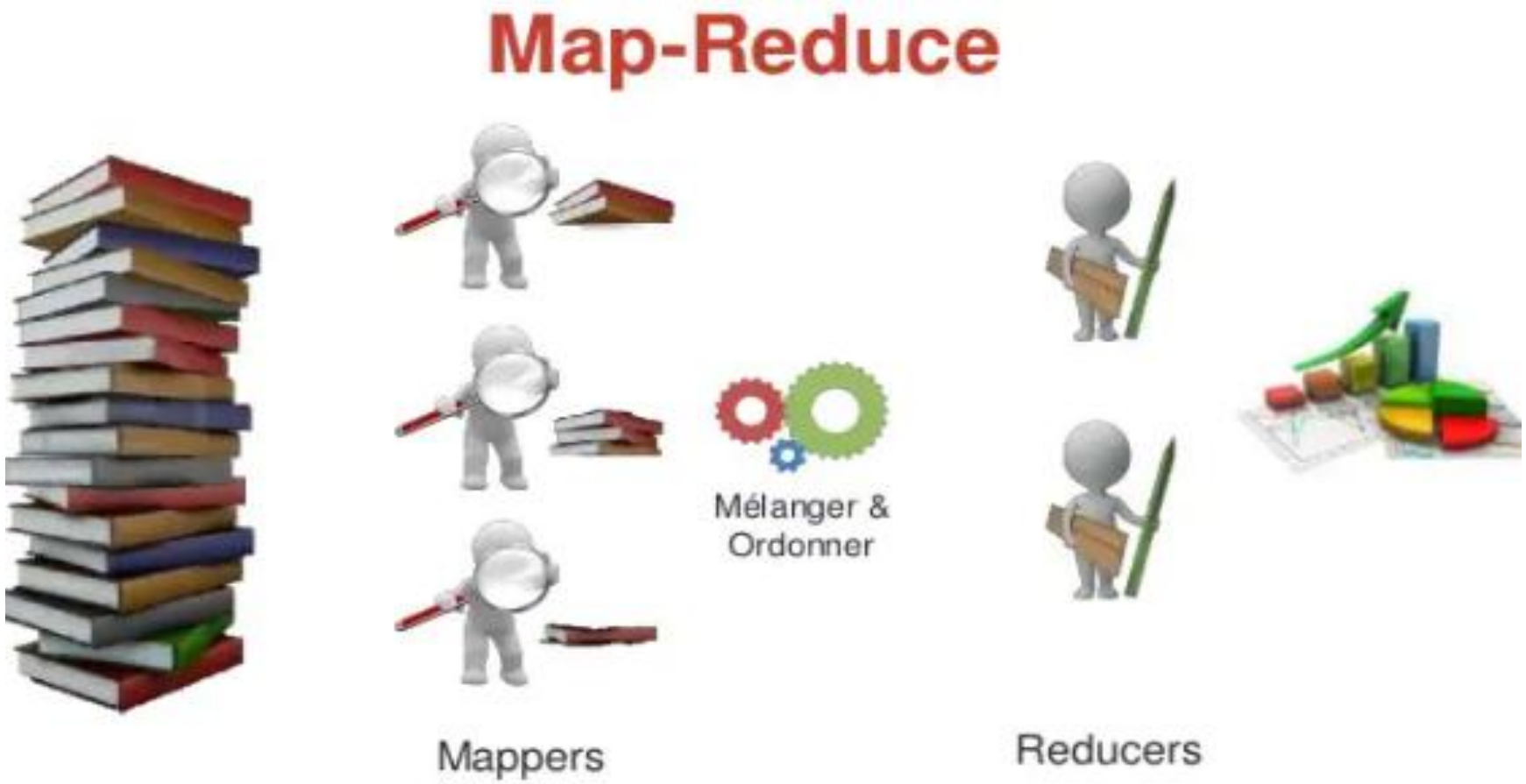
Tolérance aux Pannes

Parallélisme de Données

Répartition de Charge

Parallélisme de Traitement

Traitement des Données Massives



BIG DATA: Acteurs et Solutions

Historique : Big Data, Google : Le système de fichier GFS

Google

- Pour stocker son index grandissant quelle solution pour Google ?

1. **Utilisation d'un SGBDR ?**

   - Problème de distribution des données
   - Problème du nombre d'utilisateurs
   - Problème de Vitesse du moteur de recherche

   ✗

2. **Invention d'un nouveau système propriétaire : GFS (Google File Système) en 2003** ✓

BIG DATA: Acteurs et Solutions

Historique : Big Data, Comment exploiter ce système de fichier ?

Google

- La notion de Big Data est intimement lié à la capacité de traitements de gros volumes.

- Le premier Article a été publié en 2004 : Jeffrey Dean and Sanjay Ghemawat

➢ **MapReduce : Simplified Data Processing on Large Clusters**

- C'est un algorithme inventé par Google, Inc afin de distribuer des traitements sur un ensemble de machines avec le système GFS.

- Google possède aujourd'hui plus de 1 000 0000 de serveurs interconnectés dans le monde.

BIG DATA: Acteurs et Solutions

- Les contributeurs de l'implémentation Libre et Open Source :



- Ces entreprises ont décidé d'ouvrir leurs développements internes au monde Open Source.

- Un certains nombre de ces technologies comme « **Hadoop** » **et** « **Spark** » font partie de la fondation Apache et ont été intégrés aux offres de « **Big Data** » des grands acteurs tel que **IBM, Oracle, Microsoft, EMC** …

BIG DATA: Acteurs et Solutions

Société	Technologie développée	Type de technologie
Google	**Big Table**	Système de base de données distribuée propriétaire reposant sur GFS (Google File System). Technologie non Open Source, mais qui a inspiré Hbase qui est Open Source.
	MapReduce	Plate-forme de développement pour traitements distribués
Yahoo	**Hadoop**	Plateforme Java destinée aux applications distribuées et à la gestion intensive des données. Issue à l'origine de GFS et MapReduce.
	S4	Plateforme de développement dédiée aux applications de traitement continu de flux de données.
Facebook	**Cassandra**	Base de données de type NoSQL et distribuée.
	Hive	Logiciel d'analyse de données utilisant Hadoop.
Twitter	**Storm**	Plateforme de traitement de données massives.
	FockDB	Base de données distribuée de type graphe.
LinkedIn	**SenseiDB**	Base de données temps réel distribuée et semi-structurée.
	Voldemort	Base de données distribuée destinée aux très grosses volumétries.

Tableau : Quelques technologies Open Source du Big Data

Exemples: plateforme-technologies-outils

- Processing
  - Hadoop, Hive, Pig, mrjob, Caffeine
- NoSQL Databases
  - Hbase, MongoDB, Vertica, Cassandra, Neo4j, etc.
- Servers
  - EC2, Google App Engine, Elastic, Beanstalk, Heroku
- Analytics
  - R, SAS, Python scikit-learn, Spark MLLib, Apache Mahout
- Search
  - Solr/Lucene, ElasticSearch

Exemples: plateforme-technologies-outils



Chapitre 2: Hadoop : Un standard de gestion des données et applications distribuées

- Problématiques du calcul distribué
- La solution : Apache Hadoop
- L’écosystème Hodoop

Partie1: HDFS : Hadoop Distributed File System

- Présentation
- Architecture
- Ecriture / Lecture d’un fichier

Partie2: MapReduce

- Présentation
- Modèle de programmation
- Exemples

Chapitre 2: Hadoop : Un standard de gestion des données et applications distribuées

- **Problématiques du calcul distribué**
- **La solution : Apache Hadoop**
- **L’écosystème Hodoop**


hadoop

L’équation Hadoop

Hadoop = excellent équilibre coût / performance / stockage / flexibilité

Map Reduce

Pour pouvoir traiter un **grand volume** de données sur un cluster de machines, il faut **découper** le problème en plusieurs problèmes de taille **réduite** à exécuter sur différentes machines (stratégie « **diviser** pour **régner** »).

De multiples approches existent et ont existé pour cette division d'un problème en plusieurs « sous-tâches ».

MapReduce est un **paradigme** (modèle) de **programmation distribuée** popularisé par Google

Map Reduce

Les 4 étapes du traitement MapReduce :

- **Split**
- **Mapper**
- **Shuffle**
- **Reduce**

Map Reduce (exemple Word Count)

Objectif: compter le nombre d'occurrence de chaque mot dans un texte

1ere étape “Split”:

Première étape: déterminer une **manière de découper** (**split**) les données d'entrée pour que **chacune des machines** puisse **travailler** sur une **partie** du texte.

Notre problème est ici très simple – on peut par exemple décider de **découper** les **données d'entrée ligne par ligne**. Chacune des **lignes** du texte sera un **fragment** de nos données d'entrée.

Map Reduce (exemple Word Count)

Nos données d'entrée (le texte):

```
Celui qui croyait au ciel
Celui qui n'y croyait pas
[…]
Fou qui fait le délicat
Fou qui songe à ses querelles
```

Map Reduce (exemple Word Count)

Ce qui donne, après découpage:

```
celui qui croyait au ciel
celui qui ny croyait pas
fou qui fait le delicat
fou qui songe a ses querelles
```

on obtient **4 fragments** depuis nos données d'entrée.

Map Reduce (exemple Word Count)

Le code de notre opération MAP sera donc (ici en pseudo code):

```
POUR MOT dans LIGNE, FAIRE:
  GENERER COUPLE (MOT; 1)
```

Pour chacun de nos **fragments**, les couples **(clef; valeur)** générés seront

`celui qui croyait au ciel` → **(celui;1) (qui;1) (croyait;1) (au;1) (ciel;1)**

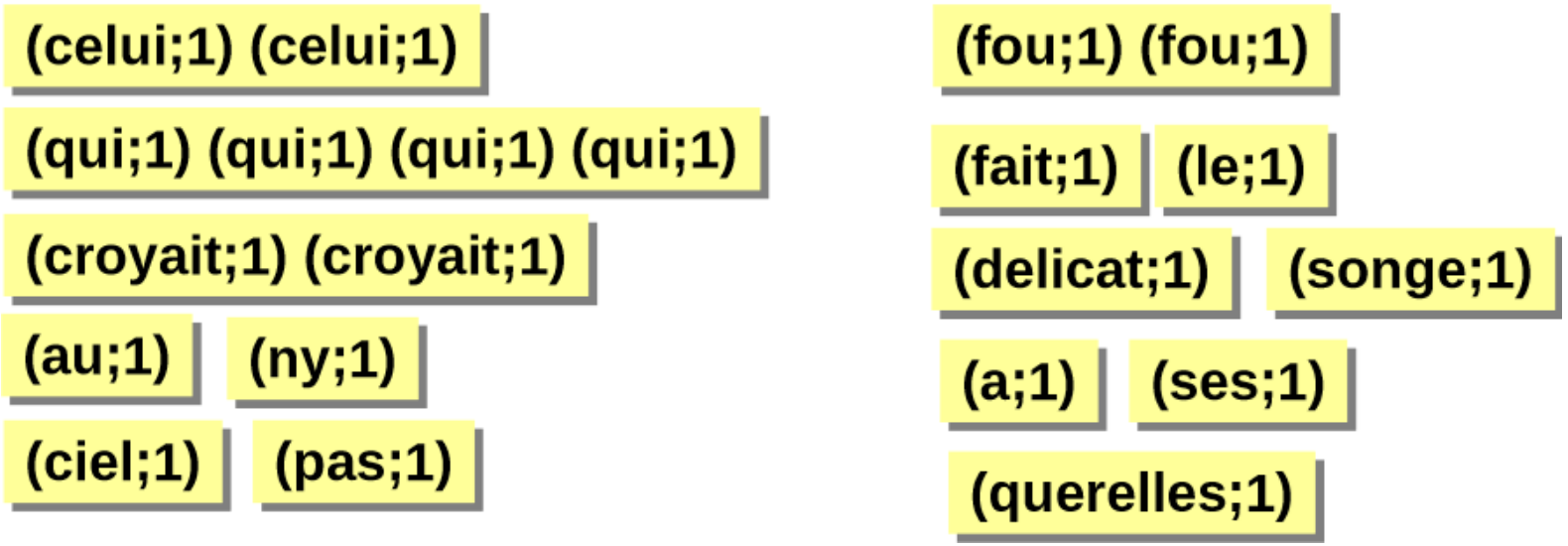
`celui qui ny croyait pas` → **(celui;1) (qui;1) (ny;1) (croyait;1) (pas;1)**

`fou qui fait le delicat` → **(fou;1) (qui;1) (fait;1) (le;1) (delicat;1)**

`fou qui songe a ses querelles` → **(fou;1) (qui;1) (songe;1) (a;1) (ses;1) (querelles;1)**

Map Reduce (exemple Word Count)

3eme étape “SHUFFLE”:



Map Reduce (exemple Word Count)

4eme étape “REDUCE”:

```
TOTAL=0
POUR COUPLE dans GROUPE, FAIRE:
   TOTAL=TOTAL+1
RENVOYER TOTAL
```

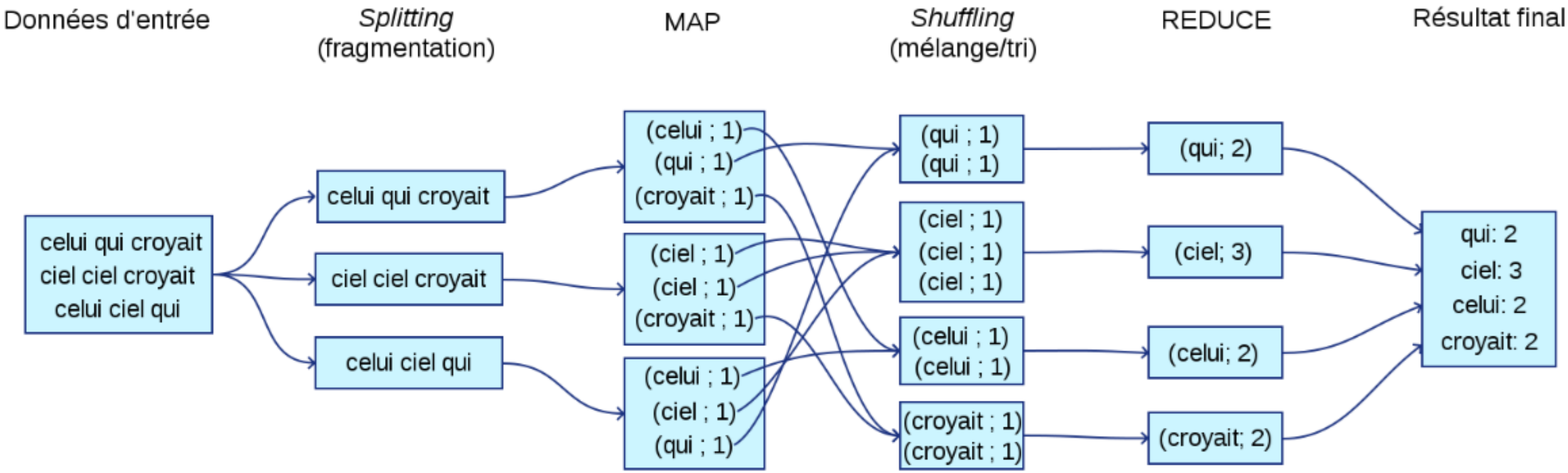
Map Reduce (exemple Word Count)

Une fois l'opération REDUCE effectuée, on obtiendra donc une **valeur unique** pour chaque **clef distincte**. En l'occurrence, notre résultat sera:

```
qui: 4
celui: 2
croyait: 2
fou: 2
au: 1
ciel: 1
ny: 1
pas: 1
fait: 1
[…]
```

Map Reduce (exemple Word Count)

Récapitulatif :



Exemple – Statistiques web

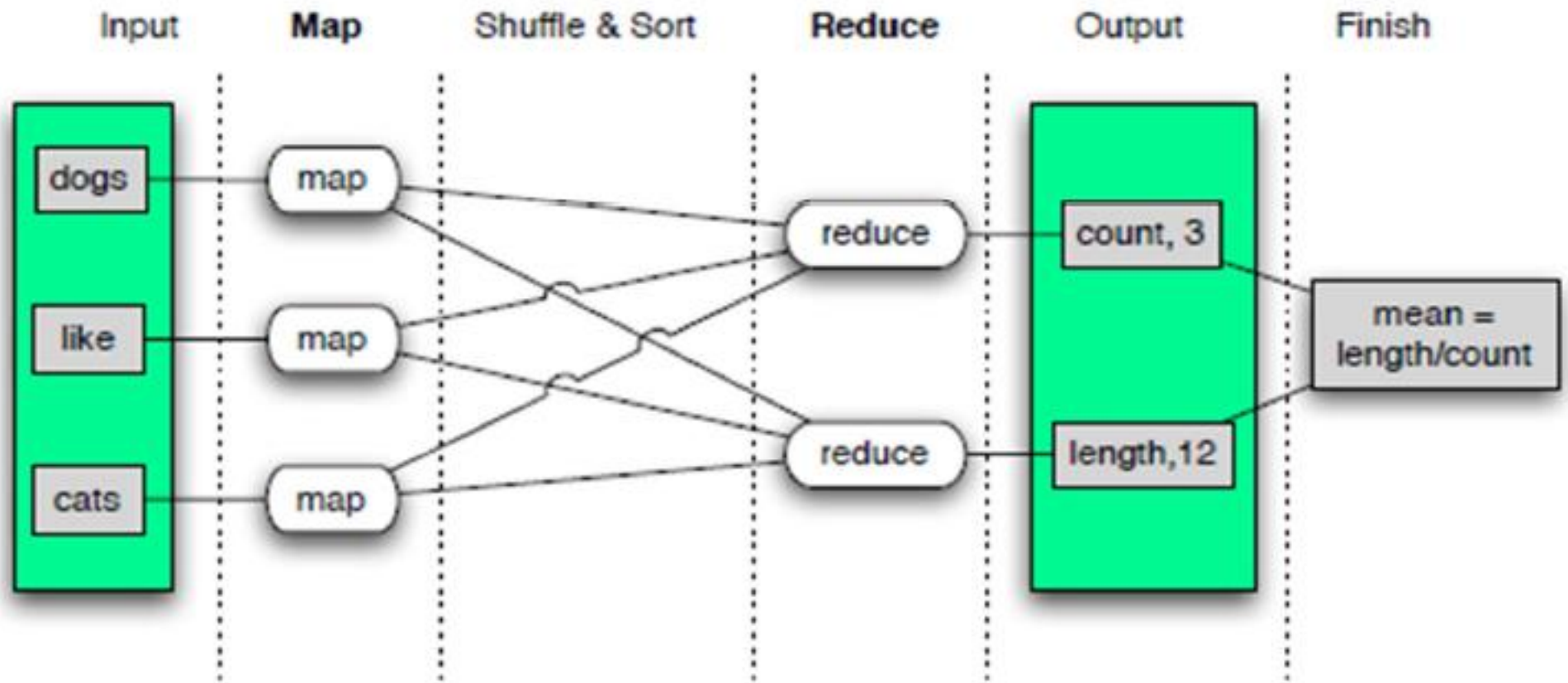
Un autre exemple: on souhaite compter le nombre de visiteurs sur chacune des pages d'un site Internet.
On dispose des fichiers de logs sous la forme suivante:

```
/index.html [19/Oct/2013:18:45:03 +0200]
/contact.html [19/Oct/2013:18:46:15 +0200]
/news.php?id=5 [24/Oct/2013:18:13:02 +0200]
/news.php?id=4 [24/Oct/2013:18:13:12 +0200]
/news.php?id=18 [24/Oct/2013:18:14:31 +0200]
...etc...
```

Exemple2 : la longueur moyenne des mots

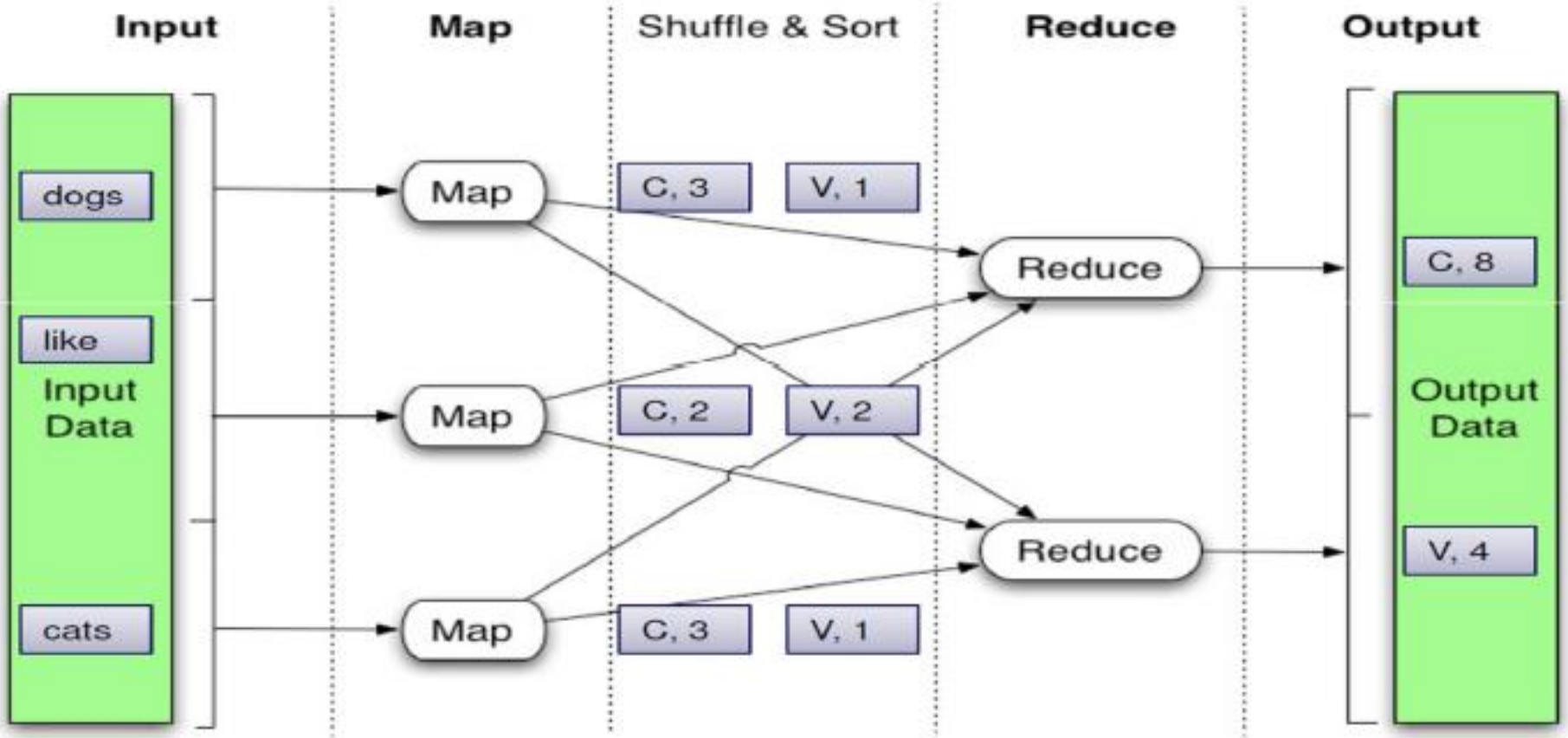
- Entrée : un grand fichier texte
- Sortie : la longueur moyenne des mots dans le fichier

Flux des données



Exemple3 : le nombre de voyelles et consonnes

Le flux des données



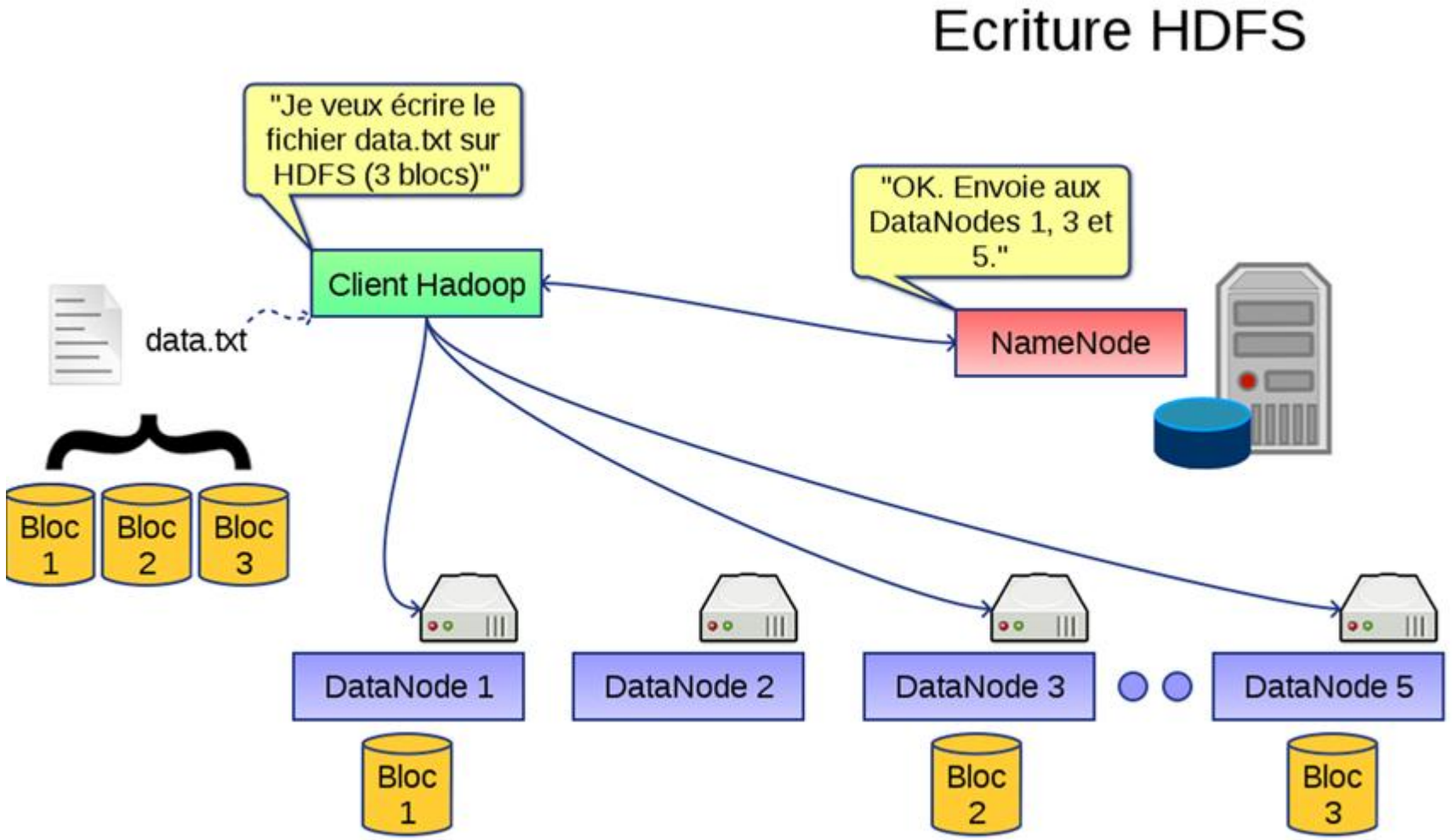
HDFS

- HDFS = Hadoop Distributed File System
- HDFS =Système de fichiers (FileSystem) d'Hadoop par défaut
- HDFS est inspiré de GFS – « The Google File System », 2003
- Principales caractéristiques :
  - **Distribué** : les données sont réparties sur les machines du cluster
  - **Répliqué** : les données sont répliquées (par défaut 3) pour palier la défaillance d'une machine
- HDFS est « taillé » pour les **fortes volumétries**

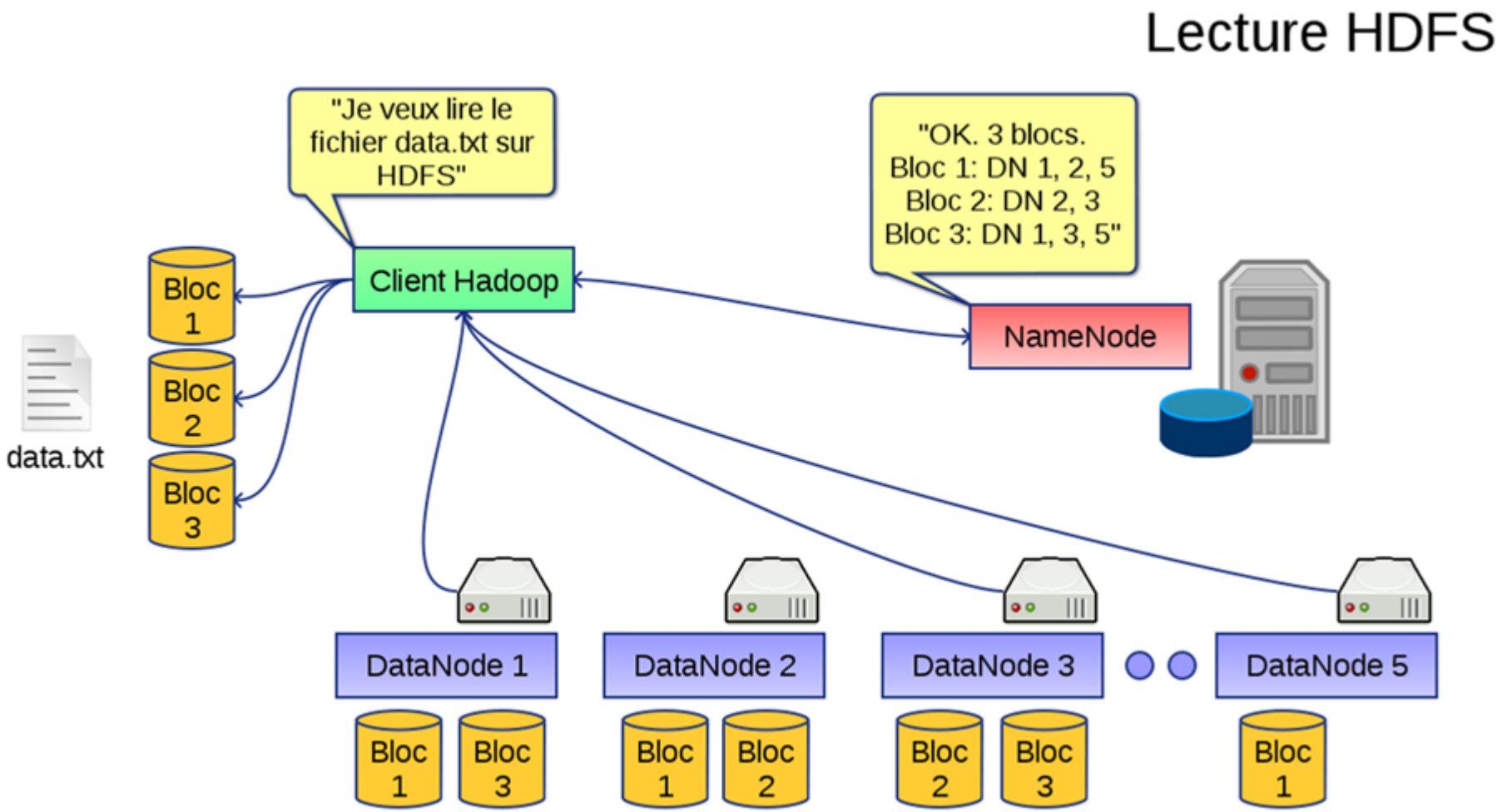
HDFS

- Dans HDFS, les données sont découpées en « **blocks** » - 128Mo par défaut
- HDFS s'appuie sur :
  - Le **NameNode** : 2 NameNodes par Cluster (Hadoop 2), et 3 NameNodes par cluster (Hadoop 3)
  - Le **DataNode** : 1 DataNode par machine du Cluster
- **Namenode** = stocke les **métadonnées** des blocks (leur localisation, taille, date de création, …)

HDFS : écriture d'un fichier



HDFS : lecture d'un fichier



HDFS : quelques commandes

hdfs dfs -ls /user

hdfs dfs -mkdir nom_dossier_hdfs

hdfs dfs -put fichier_en_local destination_sur_hdfs

hdfs dfs -copyToLocal fichier_hdfs destination_en_local

hdfs dfs -cat fichier_sur_hdfs

HDFS : quelques commandes

hdfs dfs -cp fichier_sur_hdfs destination_sur_hdfs

hdfs dfs -mv fichier_sur_hdfs destination_sur_hdfs

hdfs dfs -rm fichier_sur_hdfs

hdfs dfs -du -h dos

sier_sur_hdfs

hdfs dfs -setrep -R -w 3 fichier_sur_hdfs